

ARDUINO PHYSICAL AI CHALLENGE INDIA 2026

Project Report

Organised by Robu.in × Arduino · Submit as PDF · Max 10 MB

Project Title	Autonomous Micromouse Robot for Intelligent Maze Solving
Team Name	Team Robolution
Registration / Team ID	APC-2026-MP-87970
Contest Track	Physical AI
Institution & City	Rustamji Institute of Technology, Gwalior, Madhya Pradesh

Team Members (1–4)

Name	Email	Role
Abhishek Gupta	abhishekgupta140205@gmail.com	Team Leader

1. Project Overview

Problem Statement

Navigating an unknown maze without human intervention is a challenging robotics problem that requires accurate sensing, intelligent decision-making, and precise motion control. This project addresses this challenge by developing an autonomous Micromouse robot capable of detecting walls, exploring unknown mazes, mapping the environment, and finding the shortest path to the destination. Such technology has applications in warehouse automation, autonomous delivery robots, search-and-rescue missions, and industrial robotics.

How Your Project Works

The Micromouse robot is powered by an Arduino-based controller and uses three VL53L0X Time-of-Flight sensors to measure distances from the front, left, and right walls. Wheel encoders provide precise movement feedback, while an MPU6050 gyroscope ensures accurate 90-degree turns. A TB6612FNG motor driver controls two DC geared motors. The robot continuously reads sensor data, explores the maze, detects walls, makes navigation decisions, and uses PID control to maintain straight movement. During exploration, it creates a map of the maze and identifies the shortest route to the goal, allowing efficient autonomous navigation without human assistance.

Why Arduino UNO Q?

Arduino UNO Q provides reliable real-time processing for autonomous robotics applications. It interfaces efficiently with sensors, motor drivers, encoders, and I2C devices while executing

navigation algorithms with low latency. Its stable embedded platform makes it suitable for autonomous maze-solving applications.

2. Components Used (BOM)

List every component. The Arduino UNO Q purchase proof must be uploaded separately with your submission.

Component	Qty
Arduino UNO Q	1
TB6612FNG Motor Driver	1
VL53L0X ToF Sensor	3
MPU6050 Gyroscope	1
SSD1306 OLED Display	1
DC Gear Motors with Encoders	2
Wheels	4
Ball Caster Wheel	1
Li-ion/LiPo Battery Pack	1
Chassis	1
Piezo Buzzer	1
Status LED	1
Connecting Wires	As Required

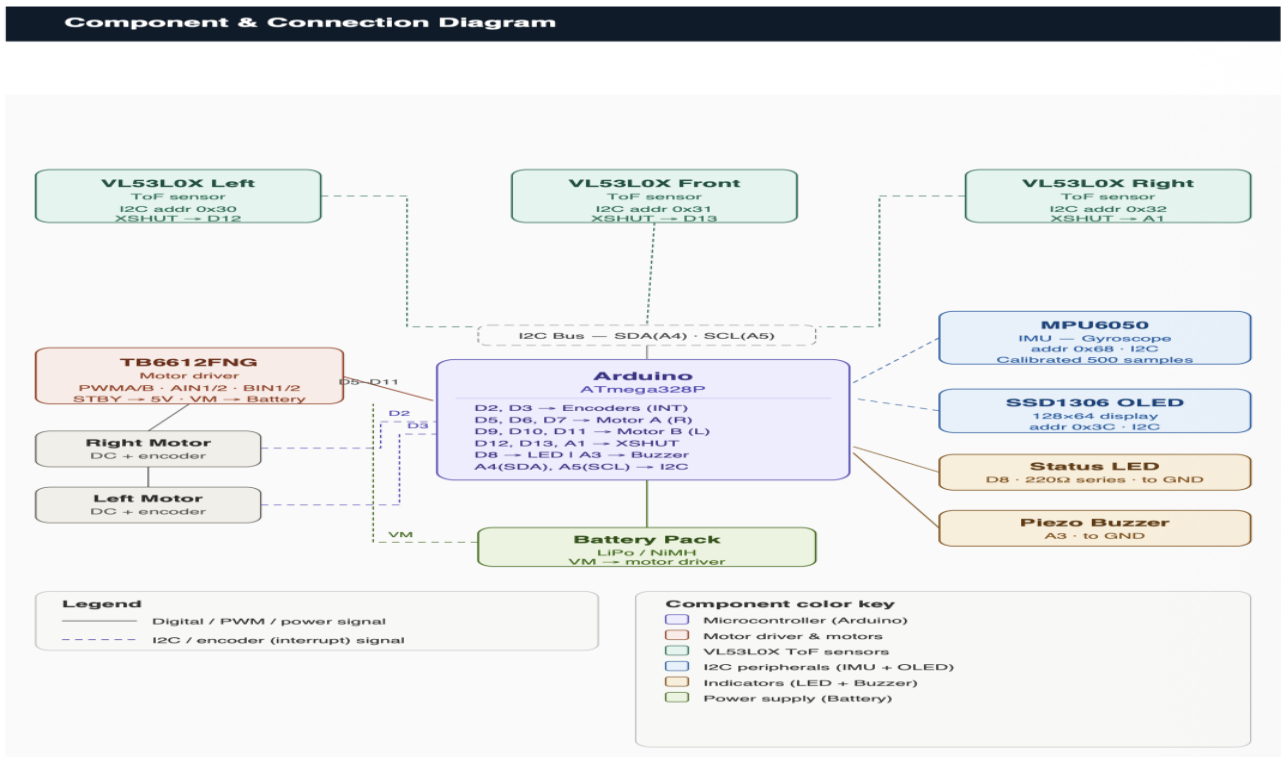
3. System Architecture & Circuit

Step-by-Step Workflow

1. Robot starts and initializes all sensors.
2. MPU6050 performs gyroscope calibration.
3. VL53L0X sensors continuously detect wall distances.
4. Arduino processes sensor data.
5. PID controller maintains straight movement.
6. If a wall is detected, the robot determines the best direction.

7. Gyroscope executes precise 90° turns.
8. Robot explores and maps the maze.
9. The shortest path is calculated.
10. Robot reaches the destination autonomously.

Block Diagram & Circuit Schematic



2.1 Motor Driver — TB6612FNG

Arduino Pin	Signal	Function
D5 (PWM)	PWMA	Right Motor speed (0–255)
D6	AIN1	Right Motor direction bit 1
D7	AIN2	Right Motor direction bit 2
D11 (PWM)	PWMB	Left Motor speed (0–255)
D9	BIN1	Left Motor direction bit 1
D10	BIN2	Left Motor direction bit 2

2.2 Quadrature Encoders (Interrupt pins)

Arduino Pin	Signal	ISR
D2 (INT0)	R_ENC_A	ISR_R — right wheel pulse count
D3 (INT1)	L_ENC_A	ISR_L — left wheel pulse count

Only channel A is used (rising edge). Direction is inferred from motor command.

2.3 VL53L0X ToF Sensors — XSHUT & I2C addresses

Arduino Pin	Signal	Sensor	Assigned I2C Addr
D12	XSHUT_L	Left	0x30
D13	XSHUT_F	Front	0x31
A1	XSHUT_R	Right	0x32
A4 (SDA)	SDA	All	Shared I2C bus
A5 (SCL)	SCL	All	Shared I2C bus

2.4 Indicators

Arduino Pin	Signal	Function
D8	LED	Status LED — blinks during turns, solid on completion
A3	BUZZER	Piezo — 2 kHz during turn, 3 kHz on completion

2.5 I2C Device Address Map

Device	I2C Address	Library
SSD1306 OLED	0x3C	Adafruit_SSD1306
MPU6050 IMU	0x68 (default)	MPU6050
VL53L0X Left	0x30 (reassigned)	VL53L0X
VL53L0X Front	0x31 (reassigned)	VL53L0X
VL53L0X Right	0x32 (reassigned)	VL53L0X

3. Circuit Description

3.1 Motor Driver Wiring – TB6612FNG

TB6612FNG Pin	Connected To	Note
VM	Battery (+)	Motor supply voltage
VCC	Arduino 5V	Logic supply
GND	Common GND	
STBY	Arduino 5V	Must be HIGH to enable
PWMA	Arduino D5	Right motor speed
AIN1 / AIN2	Arduino D6 / D7	Right motor direction
AO1 / AO2	Right motor terminals	Output wires
PWMB	Arduino D11	Left motor speed
BIN1 / BIN2	Arduino D9 / D10	Left motor direction
BO1 / BO2	Left motor terminals	Output wires

3.2 VL53L0X Sensor Wiring

Initialization order: All XSHUT LOW → enable Left (D12) → init + set 0x30 → enable Front (D13) → init + set 0x31 → enable Right (A1) → init + set 0x32.

VL53L0X Pin	Connected To	Note
VIN	Arduino 3.3V/5V	Check breakout board rating
GND	Common GND	
SDA	Arduino A4	Shared bus
SCL	Arduino A5	Shared bus – 4.7kΩ pull-up
XSHUT (Left)	Arduino D12	
XSHUT (Front)	Arduino D13	
XSHUT (Right)	Arduino A1	

3.3 MPU6050 & OLED Wiring

Device	Pin	Connected To	Note
MPU6050	VCC	Arduino 3.3V	AD0 → GND sets addr 0x68
MPU6050	SDA	Arduino A4	Shared bus
MPU6050	SCL	Arduino A5	Shared bus

Device	Pin	Connected To	Note
SSD1306	VCC	Arduino 3.3V/5V	I2C addr 0x3C
SSD1306	SDA	Arduino A4	Shared bus
SSD1306	SCL	Arduino A5	Shared bus

3.4 Encoder & Indicator Wiring

Component	Pin	Connected To	Note
Right Encoder A	OUT	Arduino D2	INT0 rising edge
Left Encoder A	OUT	Arduino D3	INT1 rising edge
LED	A	Arduino D8	220Ω series resistor to GND
Buzzer	+	Arduino A3	Piezo, negative to GND

4. Software Architecture

4.1 Main Loop Decision Flow

Step	Condition	Action
1	Every cycle	Read dF, dL, dR from ToF sensors
2	dF < 90 mm	Stop motors, wait 100 ms
3	dL > dR	Turn LEFT +90° via MPU gyro PD
4	dR > dL	Turn RIGHT -90° via MPU gyro PD
5	dF > 90 mm	moveStraightPID(255) — encoder PID

4.2 Straight-Line PID (Encoder-Based)

Parameter	Value	Role
Kp_straight	0.45	Proportional — corrects current drift
Ki_straight	0.00	Integral — disabled (avoids windup)
Kd_straight	0.25	Derivative — dampens oscillation
Error dead-band	±1 tick	Ignores tiny encoder imbalances
Correction clamp	±30 PWM	Prevents overcorrection

4.3 Turn Control — MPU6050 Gyro PD

Parameter	Value	Role
Kp_turn	1.1	Proportional turn gain
Kd_turn	0.4	Derivative — smooth approach to target
Target angle	±90°	Left or right based on open wall side
Timeout	800 ms	Safety exit if turn stalls
Settle delay	120 ms	Post-turn stabilisation

4.4 MPU6050 Calibration

On startup, 500 raw gyro Z-axis samples are averaged (~1.5 s) to calculate **gyroZOffset**. This compensates for sensor bias. Keep the robot still during this phase. The OLED shows **READY** when calibration completes.

4.5 Audio/Visual Feedback

Event	LED	Buzzer
Turning	Blinks every 100ms	2000 Hz pulses
Turn complete	ON for 150ms	3000 Hz for 150ms
Moving straight	OFF	Silent

5. PWM Calibration & Tuning Reference

The two motors have different characteristics. PWM values were tuned empirically:

Mode	Right PWM	Left PWM	Purpose
Straight	255	165	Left motor faster — reduced to balance
Turn	140	255	Counter-rotation for 90° turns

Tuning method: Run straight — reduce LEFT_PWM_MAX until path is straight. For turns — adjust RIGHT_TURN_PWM and LEFT_TURN_PWM until gyro reads exactly $\pm 90^\circ$.

5.1 Tunable Constants Quick Reference

Constant	Default	Effect if Increased
FRONT_LIMIT	90 mm	Stops farther from wall
BASE_SPEED	255	Faster straight speed
TURN_SPEED	120	Faster turn (may overshoot)
RIGHT_PWM_MAX	255	Right motor faster
LEFT_PWM_MAX	165	Left motor faster
RIGHT_TURN_PWM	140	Right faster in turns
LEFT_TURN_PWM	255	Left faster in turns
Kp_straight	0.45	More aggressive straight correction
Kd_straight	0.25	More damping
Kp_turn	1.1	More aggressive turning
Kd_turn	0.4	Smoother turn approach

6. Build Steps & Assembly

Step 1 — Chassis & Motors

3D-printed chassis (dark blue PLA). Two DC gear motors with encoders mounted at the rear. Wheels press-fit onto motor shafts. Ball caster at front for balance.

Step 2 — Motor Driver

Mount TB6612FNG on perfboard. Connect STBY to 5V. Wire AO1/AO2 to right motor, BO1/BO2 to left motor with appropriately rated wire.

Step 3 — Sensor Bracket

Three VL53LOX sensors on a forward bracket — one left, one front, one right. Mount perpendicular to maze walls at 5–10 cm height for reliable wall detection.

Step 4 — I2C Bus

Run shared SDA (A4) and SCL (A5) to all I2C devices: 3x VL53LOX, MPU6050, SSD1306. Use 4.7 k Ω pull-ups on SDA and SCL if not on breakout boards. Keep wires short.

Step 5 — Encoders

Connect encoder channel A wires to D2 (right, INT0) and D3 (left, INT1). Hardware interrupt pins ensure no missed pulses at high speed.

Step 6 — Power

Battery → TB6612FNG VM. Arduino 5V rail powers all logic ICs. Common ground for all components. 100 μ F cap across motor supply to suppress spikes.

Step 7 — Upload & Test

Upload final_code_patna.ino. On boot: OLED shows READY, gyro calibrates (~1.5 s). Place robot still on flat surface during calibration. Then enter the maze.

4. AI / ML Model Details

Model Used

Not Applicable

Training Platform

Not Applicable

Accuracy

Not Applicable

Dataset

Not Applicable

Brief Description & Limitations

This project primarily relies on sensor fusion and embedded control instead of machine learning. Navigation decisions are made using ToF sensor readings, encoder feedback, gyroscope measurements, and PID control. Performance may reduce if sensors are improperly calibrated, the battery voltage drops significantly, or highly reflective surfaces affect distance measurements.

5. Code Structure

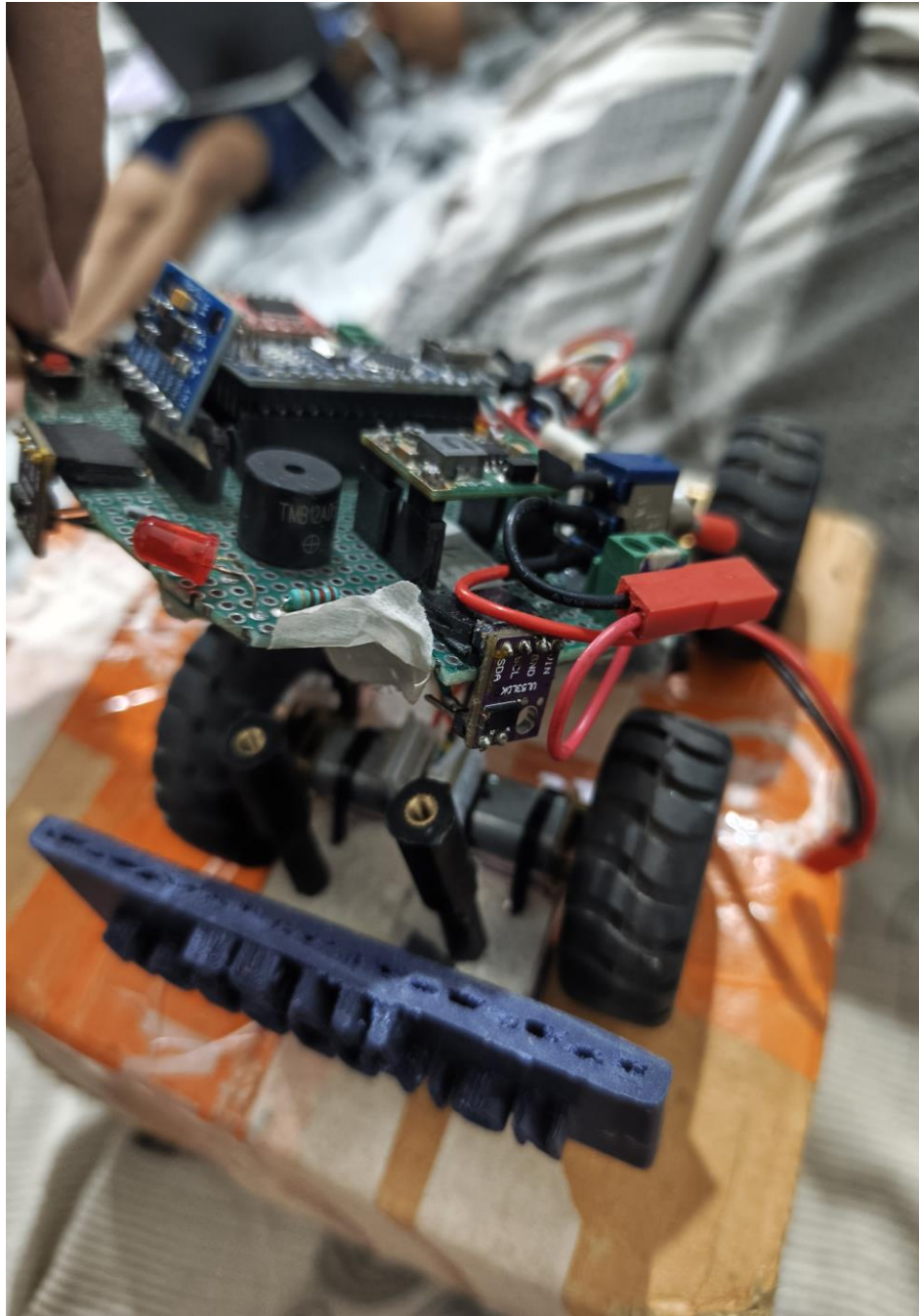
- **setup()** – Initializes sensors, OLED, motor driver, gyroscope, and encoders.
- **loop()** – Continuously reads sensor data and controls navigation.
- **moveStraightPID()** – Maintains straight movement using encoder feedback.
- **turnLeft() / turnRight()** – Executes accurate 90° turns using MPU6050.
- **readSensors()** – Reads distances from all VL53L0X sensors.
- **updateDisplay()** – Displays robot status on the OLED.
- **motorControl()** – Controls motor direction and speed.

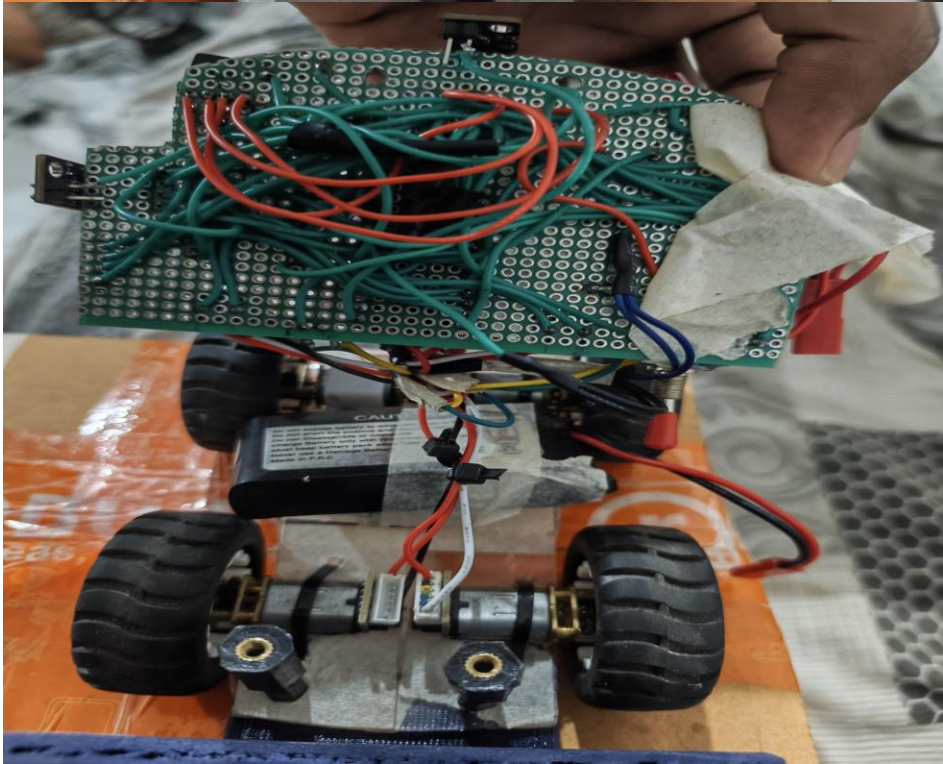
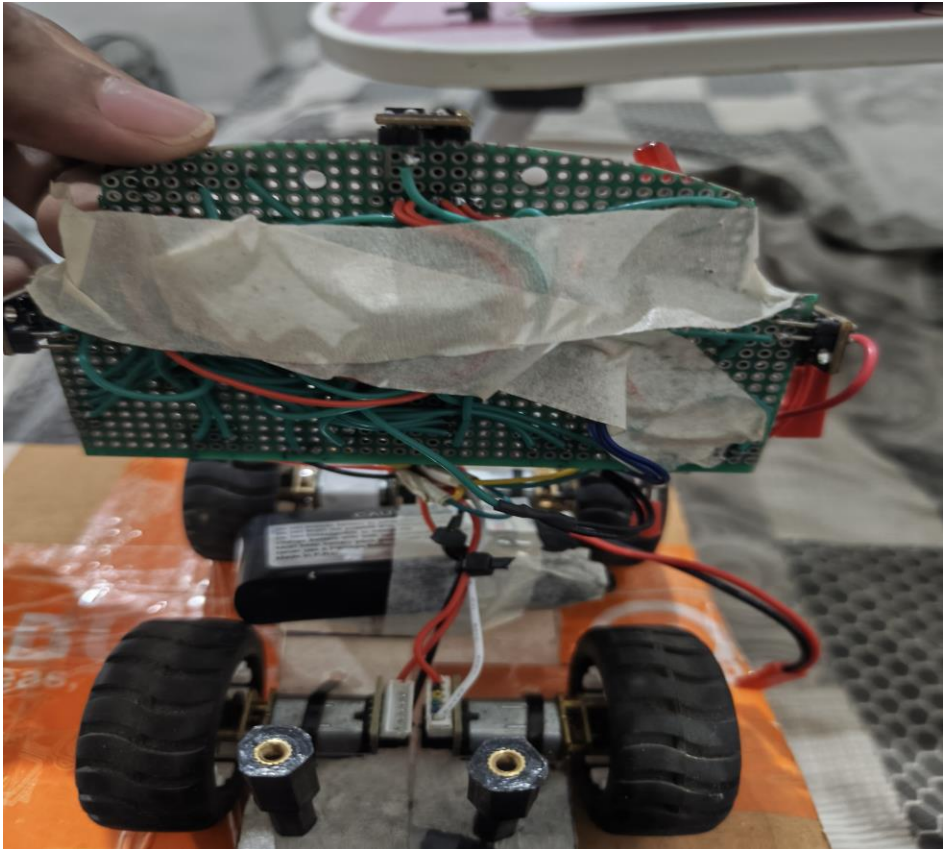
GitHub Repository	https://github.com/abhi-gupta2005/Autonomous-Micromouse-Robot
Demo Video Link	https://drive.google.com/drive/folders/14-AdEnEQPadlvJnKYnQmI1beY351CTC2?usp=sharing

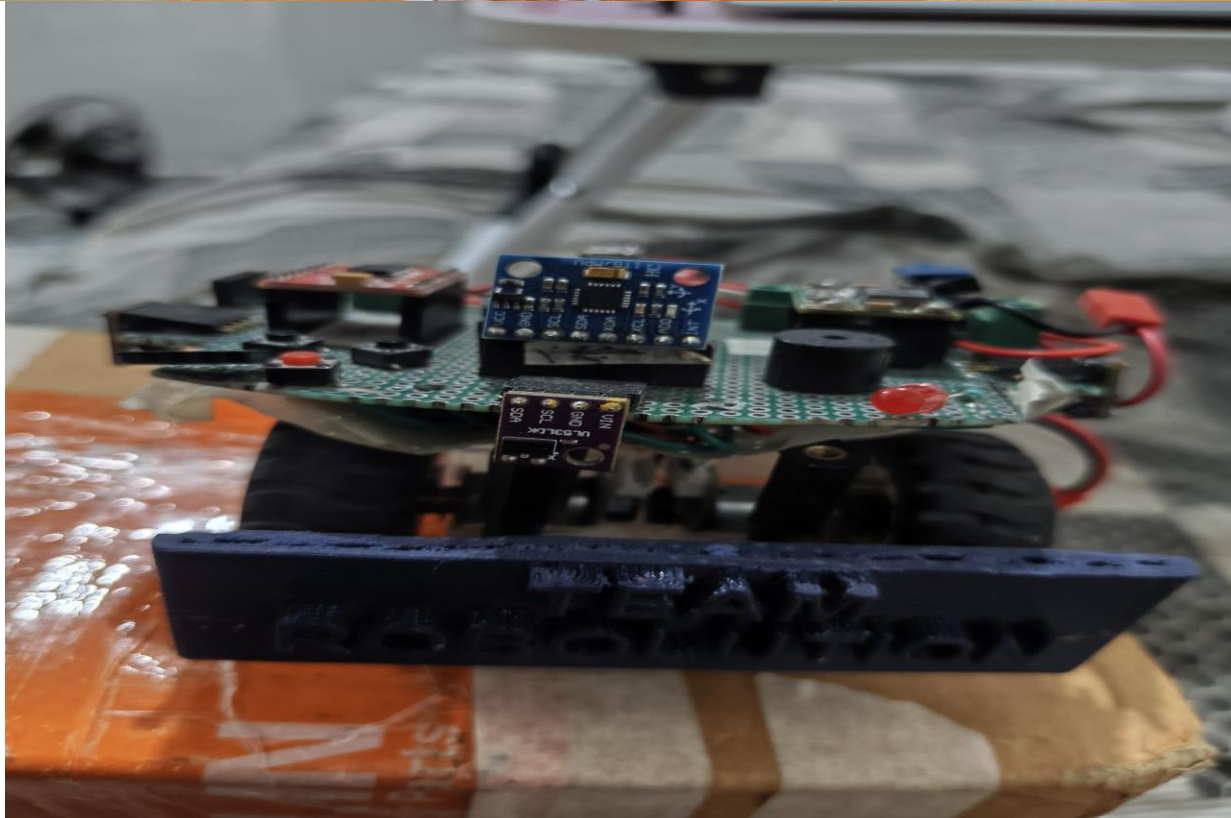
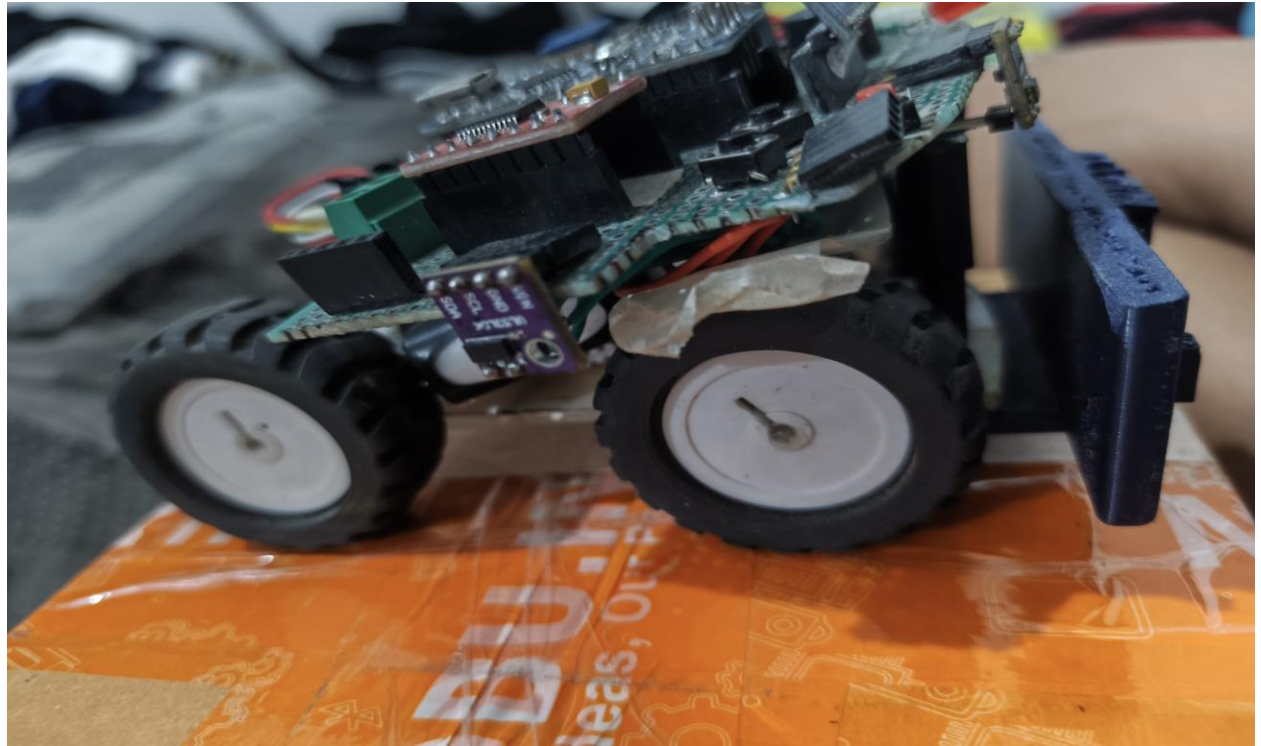
6. Testing & Results

The Micromouse robot was tested on multiple wooden maze configurations. During testing, it successfully detected walls using three VL53L0X sensors, maintained straight movement through encoder-based PID control, and executed accurate 90-degree turns using gyroscope feedback. The robot consistently navigated unknown mazes and reached the target while maintaining stable motion and reliable obstacle detection.

Project Images (2–3)







7. Challenges, Learnings & Future Improvements

Challenges Faced

- Sensor calibration
 - PID tuning
 - Gyroscope drift compensation
 - Encoder synchronization
 - Motor balancing
 - Battery power management
-

What You Learned & What's Next

This project strengthened our knowledge of embedded systems, robotics, autonomous navigation, sensor integration, motor control, and PID algorithms. Future improvements include implementing AI-based path optimization, increasing robot speed, reducing chassis weight, improving localization accuracy, and integrating wireless monitoring for real-time diagnostics.

8. Declaration

We confirm that this project is our original work and has not been published elsewhere. The Arduino UNO Q is the primary controller used in this project. All team members have reviewed and approved this report for submission.

Date	30/07/2026
------	------------